

ARQUITECTURA Y DISEÑO DEL SISTEMA (R03)

El presente capítulo describe la arquitectura global del sistema desarrollado, identificando sus principales componentes, las relaciones entre ellos y las decisiones técnicas adoptadas en su diseño. El objetivo no es detallar la implementación interna de cada módulo, sino ofrecer una visión estructurada y comprensible del sistema desde un punto de vista conceptual y técnico.

La arquitectura se ha definido antes del inicio del desarrollo, con el fin de garantizar coherencia entre los requisitos funcionales definidos en fases anteriores y la solución técnica propuesta. Este enfoque permite reducir improvisaciones, facilitar la integración de componentes y asegurar la trazabilidad entre diseño y desarrollo.

VISIÓN GENERAL DEL SISTEMA

El sistema desarrollado puede entenderse como una plataforma de gestión documental enriquecida con capacidades de búsqueda semántica y generación de respuestas contextualizadas. Desde una perspectiva de alto nivel, el sistema se compone de los siguientes bloques principales:

- Interfaz de usuario (frontend básico)
- API backend
- Sistema de almacenamiento documental
- Base de datos relacional con soporte vectorial
- Módulo de procesamiento de documentos
- Módulo de búsqueda semántica
- Flujo de generación de respuestas (RAG)

El usuario interactúa con el sistema a través de una interfaz que permita realizar operaciones como la subida de documentos, la consulta de la información y la ejecución de búsquedas. Estas peticiones son gestionadas por el backend, que actúa como núcleo de coordinación del sistema.

El backend se encarga de:

- Gestionar usuarios y autenticación
- Controlar roles y permisos
- Orquestrar el procesamiento documental
- Gestionar consultas semánticas
- Integrar el flujo de generación de respuestas

La base de datos cumple una doble función: almacenar información estructurada (usuarios, metadata, versiones) y persistir representaciones vectoriales de los fragmentos de texto procesados.

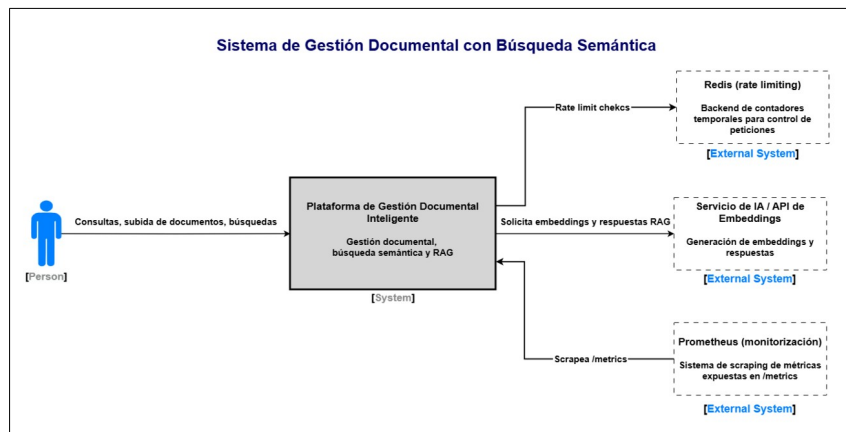


Figura 5.1 – Diagrama C4 Nivel Contexto.

A continuación, se presenta el diagrama C4 de nivel contenedor, donde se descompone el sistema en sus principales elementos desplegables y se detallan las relaciones entre ellos.

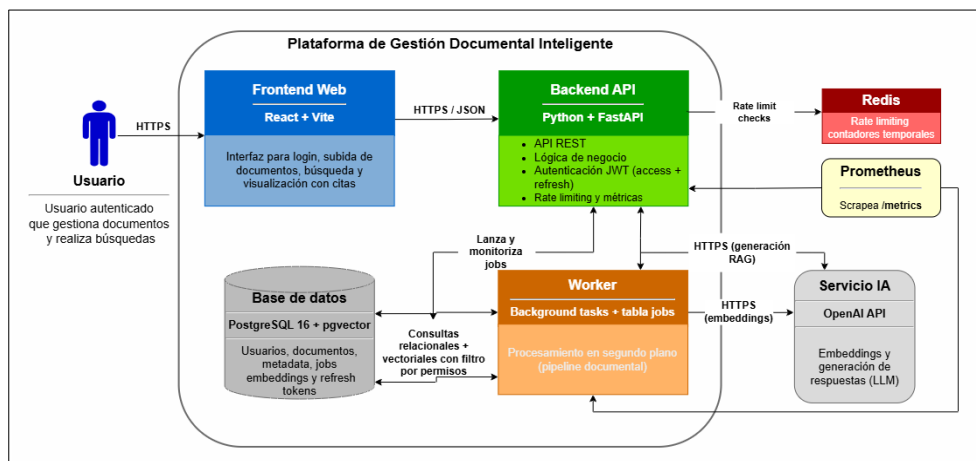


Figura 5.2 – Diagrama C4 Nivel Contenedor.

El diagrama anterior muestra la separación del sistema en contenedores desplegables claramente diferenciados. El frontend web actúa como cliente del backend, gestionando la interacción con el usuario mediante peticiones HTTPS en formato JSON. Toda la lógica de negocio y el control de acceso se centralizan en el backend API, evitando el acceso directo a la base de datos y garantizando una arquitectura controlada.

La persistencia se concentra en PostgreSQL con soporte vectorial, donde conviven los datos relacionales (usuarios, documentos, metadata y estados de procesamiento) y los embeddings

asociados a los fragmentos de texto. Esta unificación permite implementar búsquedas semánticas e híbridas sin necesidad de integrar motores externos adicionales.

El procesamiento documental se ejecuta en segundo plano mediante un componente de tipo `Worker`, encargado de la extracción de texto, fragmentación y vectorización. Este diseño evita bloquear las operaciones síncronas del backend y permite mantener trazabilidad del estado de cada trabajo. Finalmente, el servicio externo de IA se utiliza exclusivamente para la generación de embeddings y respuestas, manteniéndose fuera del dominio de la aplicación y desacoplado del resto de la infraestructura.

ARQUITECTURA LÓGICA

La arquitectura lógica del sistema se ha diseñado siguiendo un modelo de capas, con el objetivo de separar responsabilidades y facilitar el mantenimiento del código. Esta separación permite aislar cambios, mejorar la claridad estructural y reducir acoplamientos innecesarios.

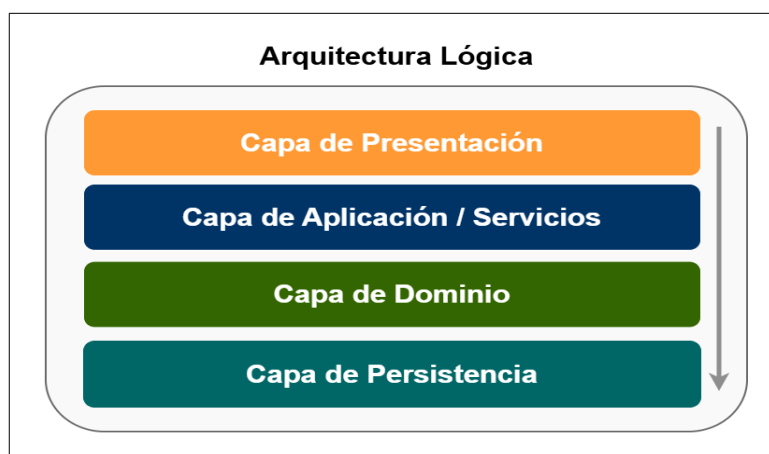


Figura 5.3 – Diagrama de Arquitectura Lógica en Capas.

Capa de presentación

Responsable de la interacción con el usuario. Incluye la interfaz básica de búsqueda y gestión documental. Su función es recoger las solicitudes del usuario y mostrar resultados, sin incorporar lógica de negocio compleja.

Capa de aplicación o servicios

Contiene la lógica principal del sistema. Aquí se gestionan los casos de uso, la coordinación entre módulos y la validación de reglas de negocio. Esta capa actúa como intermediaria entre la presentación y la persistencia.

Capa de dominio

Define las entidades principales del sistema y sus relaciones conceptuales. Incluye modelos como Usuario, Documento, Versión, Metadata, Chunk y Embedding.

Capa de persistencia

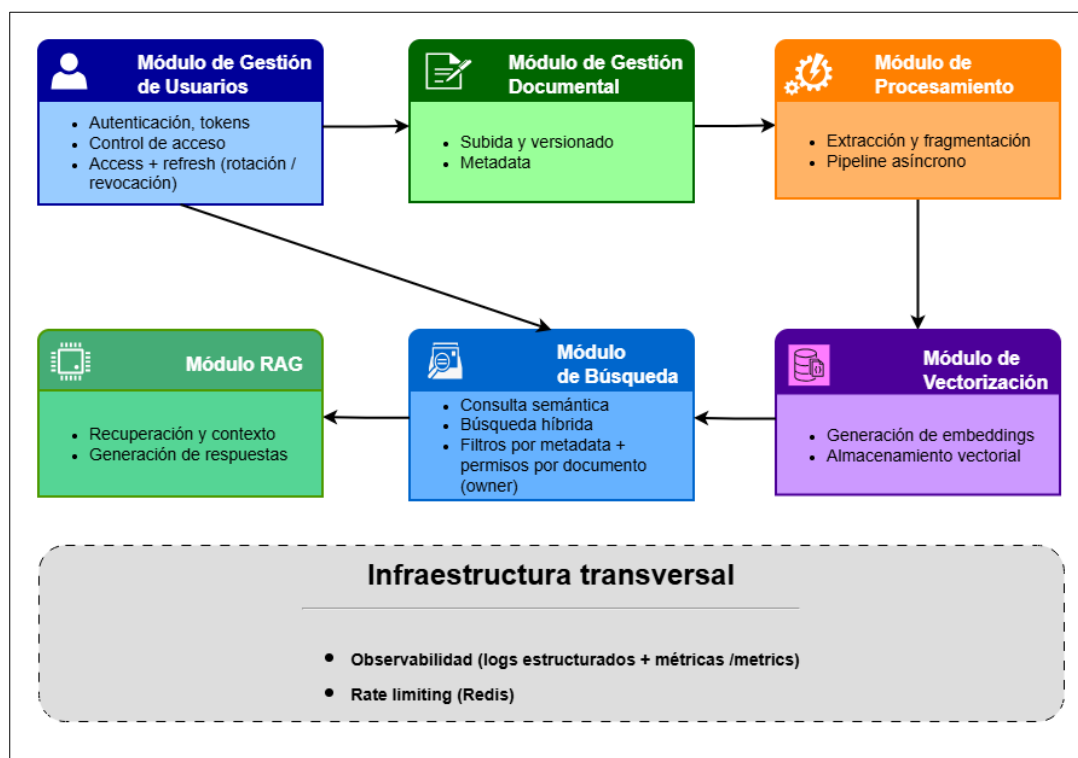
Gestiona el acceso a la base de datos, incluyendo tanto el almacenamiento relacional tradicional como el almacenamiento vectorial.

Esta arquitectura en capas permite cumplir con el principio de separación de responsabilidades, facilitando la evolución futura del sistema y evitando dependencias circulares.

COMPONENTES PRINCIPALES

La Figura 5.4 muestra donde se representan los módulos funcionales identificados y sus relaciones principales. El diagrama permite visualizar el flujo de información y la separación de responsabilidades dentro de la plataforma.

A partir de la arquitectura lógica se identifican los siguientes componentes funcionales clave del sistema:



En la Figura 5.4 – Diagrama de Componentes Principales del Sistema

MÓDULO DE GESTIÓN DE USUARIOS

Encargado de la autenticación, la generación y validación de tokens (access + refresh), el soporte de rotación y revocación de refresh tokens, y el control de acceso basado en roles. Este componente proporciona una capa transversal de seguridad, garantizando que únicamente los usuarios autorizados puedan acceder a los recursos y funcionalidades sensibles del sistema.

MÓDULO DE GESTIÓN DOCUMENTAL

Responsable de la subida de documentos, el versionado, la asociación de metadatos y la persistencia de la información estructurada. Constituye la base funcional del sistema, ya que sobre él se apoyan las capacidades de procesamiento y explotación semántica posteriores.

MÓDULO DE PROCESAMIENTO

Incluye la extracción de texto desde los documentos, su fragmentación en unidades menores (chunks) y la orquestación asíncrona del pipeline de tratamiento. Su finalidad es transformar documentos en información estructurada y preparada para su posterior análisis y vectorización.

MÓDULO DE VECTORIZACIÓN

Genera embeddings a partir de los fragmentos de texto y los almacena en la base de datos con soporte vectorial. Este componente permite representar el contenido en un espacio semántico, haciendo posible la recuperación basada en similitud.

MÓDULO DE BÚSQUEDA

Proporciona mecanismos de consulta semántica y búsqueda híbrida (texto + vector), incorporando filtrado por metadatos y restricciones de acceso por documento. Actúa como punto central de recuperación de información estructurada y vectorial, garantizando que los fragmentos devueltos respetan los permisos de usuario autenticado. En esta versión, el control de acceso se aplica por propiedad (owner), de forma que únicamente se recuperan fragmentos pertenecientes a documentos autorizados para el usuario. Modelos más avanzados como ACL o multi-tenant (tenant) se contemplan como posibles ampliaciones futuras.

MÓDULO RAG

Coordina la recuperación de fragmentos relevantes, la construcción de contexto y la generación de respuestas mediante un modelo de lenguaje. Este módulo se apoya en los anteriores sin sustituirlos, reforzando el diseño modular y la separación de responsabilidades del sistema.

DISEÑO DE DATOS

El diseño de datos se ha planteado siguiendo un modelo relacional extendido con capacidades vectoriales. Las entidades principales incluyen:

- Usuario
- Rol
- Documento
- Versión
- Metadata
- Chunk
- Embedding
- Evento (tracking)
- Refresh_Token

La Figura 5.5 muestra el diagrama simplificado del modelo de datos propuesto, donde se representan las entidades principales del sistema y las relaciones existentes entre ellas.

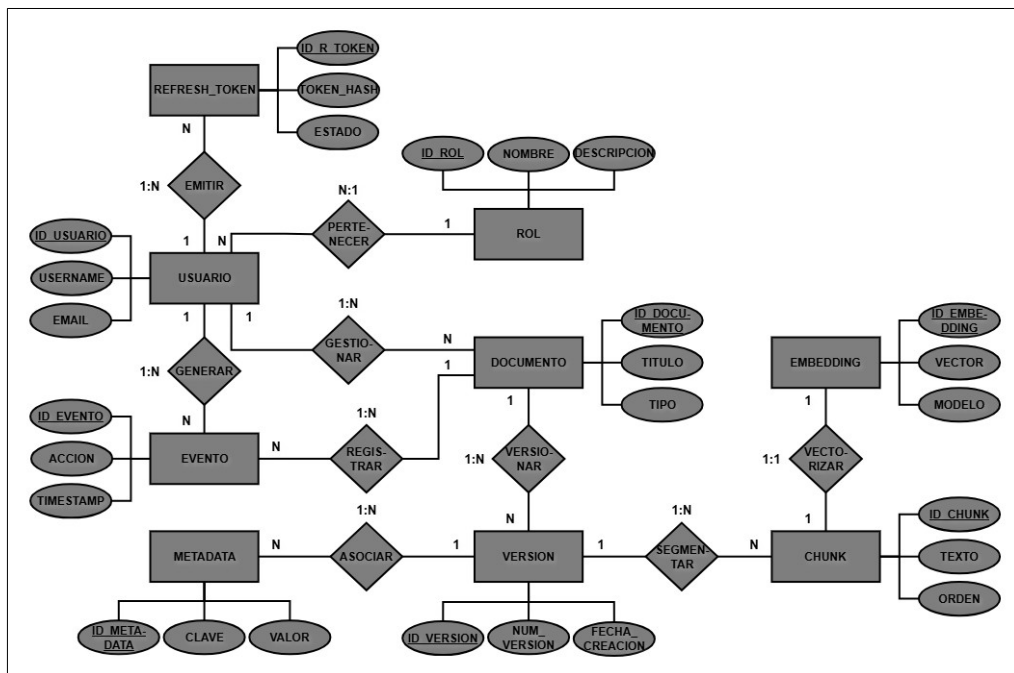


Figura 5.5 – Diagrama de Modelo de Datos E/R

DECISIONES TÉCNICAS ADOPTADAS

La selección tecnológica del proyecto se ha realizado atendiendo a criterios de coherencia arquitectónica, viabilidad de implementación en un contexto académico y alineación con tecnologías ampliamente utilizadas en entornos profesionales actuales. No se ha buscado incorporar herramientas por tendencia o novedad, sino aquellas que permiten cumplir los requisitos definidos de manera estructurada, mantenible y justificable.

La arquitectura propuesta combina un backend moderno orientado a servicios, una base de datos relacional con capacidades vectoriales, un pipeline de procesamiento documental modular y un entorno de despliegue reproducible. Las decisiones adoptadas buscan equilibrio entre robustez técnica y complejidad razonable para un desarrollo individual.

A continuación, se detallan las principales decisiones técnicas adoptadas, justificando su elección y el descarte de las alternativas consideradas.

Tabla 5.1 – Tecnologías seleccionadas y alternativas descartadas

Área	Tecnología elegida	Por qué esta	Alternativa descartada
Backend API	Python + FastAPI	Ecosistema IA y rapidez de desarrollo	Java + Spring Boot
Servidor ASGI	Uvicorn + Gunicorn	Entorno productivo y compatible con contenedores	Servidor embebido simple
Base de datos	PostgreSQL 16	Robustez relacional y extensibilidad	MongoDB
Búsqueda vectorial	pgvector	Unificación relacional + vectorial	Motor vectorial externo
ORM	SQLAlchemy 2.0 + Alembic	Control de modelo y migraciones versionadas	Acceso directo sin ORM
Procesamiento documental	PyMuPDF + python-docx	Extracción fiable y ligera	OCR completo
Autenticación	JWT (access + refresh) + RBAC	Seguridad stateless con renovación segura	Sesiones en servidor
Pipeline asíncrono	Background tasks + tabla jobs	Simplicidad sin infraestructura adicional	Celery + Redis
Rate limiting	Redis	Contadores rápidos y atómicos sin cargar la BD	Rate limiting en PostgreSQL
Embeddings / IA	OpenAI API	Integración estable y rápida	Modelo local autoalojado
Frontend	React + Vite	Ligero y suficiente para validación funcional	Framework SSR complejo
Contenerización	Docker Compose	Reproducibilidad y coherencia DevOps	Instalación manual

Testing	pytest	Ecosistema consolidado en Python	Tests ad-hoc
Control de versiones	Git + GitHub	Trazabilidad y control de desarrollo	Control local sin remoto

Nota:

BACKEND Y ARQUITECTURA DE SERVICIOS

El backend se ha implementado utilizando Python y el framework FastAPI, orientado a la construcción de APIs REST modernas. Esta elección permite estructurar el sistema de endpoints claramente definidos, favoreciendo la separación de responsabilidades y la modularidad del código.

FastAPI ofrece validación automática de datos mediante modelos tipados y generación automática de documentación OpenAPI, lo que facilita tanto el desarrollo como la validación funcional del sistema. Su integración natural en el ecosistema Python resulta especialmente adecuada para proyectos que incorporan procesamiento de texto y técnicas de inteligencia artificial.

La alternativa Java con Spring Boot se consideró desde un punto de vista empresarial; sin embargo, se descartó por introducir una mayor complejidad estructural sin aportar ventajas significativas para el alcance académico del proyecto.

PERSISTENCIA Y MODELO DE DATOS

La base de datos seleccionada es PostgreSQL, complementada con la extensión pgvector para el almacenamiento de embeddings. Esta decisión permite mantener tanto los datos estructurados (usuarios, documentos, versiones, metadata) como las representaciones vectoriales en un único sistema de persistencia.

Esta unificación simplifica la arquitectura y facilita la trazabilidad entre documentos y resultados de búsqueda semántica. Además, PostgreSQL proporciona robustez transaccional y un modelo relacional coherente con el diseño de datos definido en el apartado anterior.

Se descartó el uso de motores externos de búsqueda o bases de datos vectoriales independientes, ya que introducirían complejidad adicional y fragmentarían la arquitectura sin aportar un beneficio proporcional en el contexto del proyecto.

SEGURIDAD Y CONTROL DE ACCESO

La autenticación se basa en tokens JWT con un modelo de control de acceso basado en roles (RBAC), manteniendo una arquitectura stateless y protegiendo los endpoints del sistema de forma granular.

Para reforzar la seguridad y permitir sesiones persistentes sin recurrir a estado en servidor, se incorpora un esquema de access token de vida corta (10-20 minutos) y refresh token de vida más larga (7-30 días). El refresh token permite obtener nuevos access tokens sin requerir un login completo.

Con el fin de mitigar riesgos de reutilización indebida, se aplica rotación de refresh tokens: cada refresh válido utilizado se invalida y se emite uno nuevo. Además, se contempla revocación (logout o sospecha), almacenando en base de datos el refresh token de forma segura mediante hash, junto con información de estado (revocado, reemplazado o expirado).

Si bien el RBAC controla el acceso a funcionalidades y endpoints, el sistema aplica también el principio de mínimo privilegio en el acceso a la información documental: en el contexto de búsqueda semántica y RAG, la recuperación de chunks se filtra en función de los permisos sobre documentos, garantizando que un usuario solo pueda recuperar información de documentos autorizados.

PROCESAMIENTO DOCUMENTAL Y PIPELINE

El procesamiento de documentos se implementa mediante librerías especializadas en extracción de texto, integradas dentro de un pipeline modular. Se ha optado por un modelo de tareas en segundo plano gestionado mediante funciones asíncronas y una tabla de seguimiento de estados.

Esta solución permite mantener un flujo controlado sin introducir dependencias adicionales como sistemas de mensajería externos. Aunque herramientas como Celery y Redis podrían aportar mayor escalabilidad, se consideró que excedían el alcance y complejidad razonable del proyecto.

VECTORIZACIÓN E INTEGRACIÓN DE IA

Para la generación de embeddings y el flujo RAG se ha optado por integrar un modelo accesible mediante API externa. Esta decisión permite centrarse en la arquitectura de recuperación y control de contexto sin asumir la gestión de infraestructura de modelos locales.

Se ha priorizado la estabilidad, documentación y facilidad de integración frente a soluciones experimentales o autoalojadas que aumentarían significativamente la carga técnica.

ENTORNO DE EJECUCIÓN Y CALIDAD

El sistema se despliega mediante Docker Compose, lo que garantiza reproducibilidad y coherencia entre entornos. El uso de contenedores facilita la evaluación del proyecto y refuerza la alineación con prácticas profesionales actuales.

Se han incorporado pruebas automatizadas mediante pytest y control de versiones mediante Git y repositorio remoto, asegurando trazabilidad del desarrollo. Además, se incorporan mecanismos de observabilidad para facilitar el análisis del comportamiento del sistema. Se generan logs estructurados en formato JSON incluyendo (si aplica), endpoint, código de respuesta y latencia, así como información por etapa del pipeline (retrieval, embedding, LLM). Asimismo, se expone un endpoint compatible con Prometheus para el registro de métricas de peticiones HTTP y del flujo RAG.

Para proteger recursos y evitar abuso del sistema, se implementa rate limiting sobre endpoints críticos como y devolviendo HTTP 429 cuando supera el umbral configurado.

CIERRE DEL APARTADO

Las decisiones técnicas adoptadas reflejan un enfoque equilibrado entre solidez arquitectónica, viabilidad académica y alineación con tecnologías demandadas en entornos profesionales actuales. Cada elección responde a una necesidad concreta del sistema y se integra de forma coherente en la arquitectura global definida en este capítulo.

Asimismo, todas las tecnologías seleccionadas permiten una evolución futura del sistema sin necesidad de rediseñar la arquitectura base.